۱-۴: عملیات سیستم عامل

پس از آشنایی با سخت‌افزار و معماری‌های مختلف، اکنون به قلب ماجرا می‌پردازیم: **سیستم‌عامل‌چگونه‌کارمی‌کند؟** در این بخش، فرآیندهای بنیادینی را که یک سیستم‌عامل برای مدیریت سیستم و اجرای برنامه‌ها به کار می‌گیرد، کالبدشکافی می‌کنیم.

۱-۴-۱: راه‌اندازی سیستم (Booting) - لحظه تولد

فرآیند روشن شدن یک کامپیوتر تا آماده به کار شدن آن، یک سکانس دقیق و مهندسی شده به نام **بوت‌شدن** است.

1.**اجرای‌برنامه‌بالاآورنده(BootstrapProgram):** با فشردن دکمه پاور، اولین کدی که اجرا می‌شود، برنامه بالاآورنده است. این برنامه ساده در یک حافظه دائمی و فقط خواندنی به نام **سفت‌افزار(Firmware)** (مانند ROM یا EEPROM روی مادربورد) ذخیره شده است.

2.**مقداردهی‌اولیه‌سخت‌افزار:** برنامه بالاآورنده یک بررسی اولیه روی تمام سخت‌افزار سیستم انجام داده و آن‌ها را مقداردهی اولیه می‌کند (ثبات‌های CPU، کنترلرهای دستگاه، حافظه و غیره).

3.**یافتن‌وبارگذاری‌هسته(Kernel):** مهم‌ترین وظیفه برنامه بالاآورنده، یافتن **هسته(Kernel)** سیستم‌عامل از یک مکان از پیش‌تعیین‌شده در حافظه ثانویه (مانند بوت سکتور دیسک) و بارگذاری آن در حافظه اصلی (RAM) است.

4.**شروع‌به‌کارهسته:** پس از بارگذاری، کنترل سیستم به هسته واگذار می‌شود. هسته شروع به اجرا کرده و اولین پردازه سیستمی (مانند `init` در یونیکس‌های قدیمی یا `systemd` در لینوکس مدرن) را راه‌اندازی می‌کند.

5.**اجرای** گماشته‌های**سیستمی(SystemDaemons):** این پردازه اولیه، سایر سرویس‌ها و **گماشته‌های‌سیستمی** (برنامه‌هایی که در پس‌زمینه به طور مداوم اجرا می‌شوند) را فعال می‌کند.

6.**آماده‌به‌کار:** پس از تکمیل این مراحل، سیستم به طور کامل راه‌اندازی شده و در حالت آماده‌باش برای دریافت یک **رویداد (Event)**، که معمولاً از طریق یک **وقفه(Interrupt)** اعلام می‌شود، قرار می‌گیرد.

۱-۴-۲: چندبرنامگی و چندوظیفگی (Multiprogramming &**Multitasking**)

این دو مفهوم، انقلابی در استفاده بهینه از منابع کامپیوتر ایجاد کردند و اساس سیستم‌عامل‌های مدرن هستند.

چندبرنامگی (Multiprogramming):

هدف‌اصلی: افزایش بهره‌وری CPU (CPU**Utilization**).

ایده‌کلیدی: یک برنامه به ندرت می‌تواند CPU را به طور صد در صد مشغول نگه دارد، زیرا اغلب منتظر عملیات ورودی/خروجی (I/O) می‌ماند. در یک سیستم تک‌برنامه‌ای، این زمان‌های انتظار به معنای بیکار ماندن و هدر رفتن توان CPU است.

راه‌حل: سیستم‌عامل چندین **پردازه(Process)** - یعنی برنامه در حال اجرا - را به طور همزمان در حافظه نگه می‌دارد. هرگاه یک پردازه برای یک رویداد طولانی (مانند خواندن از دیسک) منتظر بماند، سیستم‌عامل به سرعت CPU را به پردازه دیگری که آماده اجراست، اختصاص می‌دهد. این کار تضمین می‌کند که CPU تا جای ممکن **همیشه‌کاری‌برای‌انجام دادن‌دارد**.

چندوظیفگی (Multitasking) یا اشتراک زمانی (Time-sharing):

هدف‌اصلی: فراهم کردن یک **سیستم‌تعاملی(InteractiveSystem)** با زمان پاسخ‌دهی سریع برای کاربر.

ایده‌کلیدی: این تکنیک، بسط منطقی چندبرنامگی است. در این حالت، CPU به قدری سریع بین پردازه‌ها جابجا می‌شود (معمولاً در بازه‌های زمانی کوتاه چند میلی‌ثانیه‌ای به نام **کوانتوم‌زمانی**) که کاربر تصور می‌کند چندین برنامه به طور همزمان در حال اجرا هستند.

مثال: زمانی که شما در حال تایپ در یک واژه‌پرداز هستید، سرعت تایپ شما برای کامپیوتر بسیار کند است. در فواصل بین فشردن کلیدها، سیستم‌عامل صدها بار CPU را به برنامه‌های دیگر (مانند پخش موسیقی در پس‌زمینه یا بررسی ایمیل‌ها) اختصاص داده و دوباره به واژه‌پرداز بازمی‌گرداند، بدون اینکه شما متوجه این جابجایی‌ها شوید.

نیازمندی‌ها: پیاده‌سازی چندوظیفگی نیازمند مکانیزم‌های پیچیده‌ای مانند **زمان‌بندیCPU(CPUScheduling)**، **مدیریت حافظه، همگام‌سازی‌پردازه‌ها** و **حافظه‌مجازی(VirtualMemory)** است.

حافظه‌مجازی(VirtualMemory):یک تکنیک قدرتمند که به پردازه‌ها اجازه می‌دهد بزرگتر از حافظه فیزیکی واقعی (RAM) باشند و به طور کامل در حافظه قرار نگیرند. این کار با جداسازی **فضای‌آدرس‌منطقی** کاربر از **فضای‌آدرس فیزیکی**، امکان اجرای همزمان برنامه‌های بیشتر و بزرگتر را فراهم کرده و برنامه‌نویسی را ساده‌تر می‌کند.

۳-۴-۱: عملیات مد دوگانه و حفاظت (Dual-Mode Operation)

این مفهوم، سنگ بنای **حفاظت‌وامنیت** در سیستم‌عامل است و از سیستم‌عامل در برابر برنامه‌های کاربر مخرب یا خطادار محافظت می‌کند.

ایده‌کلیدی: سخت‌افزار حداقل دو مد عملیاتی مجزا را پشتیبانی می‌کند:

1.**مدکاربر(UserMode):** برنامه‌های کاربردی عادی در این مد اجرا می‌شوند. در این حالت، دسترسی به سخت‌افزار و دستورالعمل‌های حساس، **محدودوکنترل‌شده** است.

2.**مدهسته(KernelMode):** که به آن مد سوپروایزر، مد سیستم یا مد ممتاز نیز می‌گویند. خود سیستم‌عامل (هسته) در این مد اجرا می‌شود و **دسترسی‌کامل‌ونامحدودی** به تمام سخت‌افزار و دستورالعمل‌ها دارد.

بیت‌مد(ModeBit): یک بیت در سخت‌افزار CPU که مد فعلی را مشخص می‌کند (معمولاً **0برای‌هسته** و **1برای‌کاربر**)

دستورالعمل‌های‌ممتاز(PrivilegedInstructions):دستورالعمل‌های حساسی که می‌توانند به سیستم آسیب بزنند و اجرای آن‌ها تنها در **مدهسته** مجاز است. اگر برنامه‌ای در مد کاربر تلاش کند یکی از این دستورالعمل‌ها را اجرا کند، سخت‌افزار یک خطای **تله(Trap)** ایجاد کرده و کنترل را به سیستم‌عامل می‌دهد.

مثال‌های‌مهم:

1.دستورالعمل تغییر به مد هسته.

2.دستورالعمل‌های کنترل مستقیم ورودی/خروجی

. 3.دستورالعمل‌های مدیریت تایمر

.4.دستورالعمل‌های مدیریت وقفه

.5.دستورالعمل‌های دستکاری جداول مدیریت حافظه.

فرآیند انتقال بین مدها:

1.سیستم با **مدهسته** راه‌اندازی می‌شود.

2.سیستم‌عامل یک برنامه کاربر را با تغییر بیت مد به **مدکاربر**، اجرا می‌کند

3.زمانی که برنامه کاربر نیاز به سرویسی از سیستم‌عامل دارد (مانند خواندن از فایل)، یک **فراخوانی‌سیستمی(System Call)** را اجرا می‌کند.

4.فراخوانی سیستمی باعث وقوع یک **تله** شده، سخت‌افزار بیت مد را به **مدهسته** تغییر می‌دهد و کنترل را به یک بخش مشخص از کد سیستم‌عامل منتقل می‌کند.

5.سیستم‌عامل درخواست را در مد هسته (با دسترسی کامل) انجام می‌دهد.

6.پس از اتمام کار، سیستم‌عامل بیت مد را دوباره به **مدکاربر** تغییر داده و کنترل را به برنامه کاربر بازمی‌گرداند.

چندمدی(Multi-Mode): برخی پردازنده‌های مدرن (مانند اینتل) بیش از دو مد (مثلاً چهار حلقه حفاظتی) فراهم می‌کنند تا بتوان سطوح دسترسی متفاوتی را برای اجزای مختلف سیستم‌عامل یا درایورها تعریف کرد.

۱-۴-۴: تایمر (Timer) - نگهبان زمان

برای جلوگیری از اینکه یک برنامه کاربر (به دلیل خطا یا یک حلقه بی‌نهایت) کنترل CPU را برای همیشه در انحصار خود بگیرد و سیستم را قفل کند، از یک **تایمرسخت‌افزاری** استفاده می‌شود.

نحوه عملکرد:

1.سیستم‌عامل قبل از دادن کنترل به یک برنامه کاربر، تایمر را برای یک دوره زمانی مشخص (یک کوانتوم زمانی) تنظیم می‌کند

2.برنامه کاربر شروع به اجرا می‌کند

3.اگر برنامه قبل از اتمام این زمان، کنترل را به صورت داوطلبانه (مثلاً با یک فراخوانی سیستمی) پس ندهد، تایمر به صفر رسیده و یک **وقفه(TimerInterrupt)** ایجاد می‌کند.

4.این وقفه، کنترل را به طور **خودکارواجباری** به سیستم‌عامل بازمی‌گرداند.

5.سیستم‌عامل سپس می‌تواند تصمیم بگیرد که به برنامه زمان بیشتری بدهد یا آن را خاتمه دهد.

حفاظت: دستورالعمل‌هایی که مقدار تایمر را تغییر می‌دهند، **ممتاز** هستند و فقط در مد هسته قابل اجرا می‌باشند تا برنامه‌های کاربر نتوانند